

Game Engine Course Assignments – Self-Evaluation

Student Name: Haolun Yuan

Project Title: GameExample

Date: 2025/08/13

1. Collision Mechanics

I, Haolun Yuan, deserve a mark of **95%** for collision mechanics.

I demonstrate competency in this area in the game I made by:

Engine-Side Implementation:

- Designed and implemented a robust AABB collision detection system integrated with the ECS architecture, using efficient registry queries via `entt`.
- Built a layer-based collision filtering system to control which entity types can interact (e.g., Player vs Enemy).
- Created a deterministic collision resolution pipeline that handles overlaps without tunneling issues.
Developed a flexible callback mechanism for systems to register and respond to collision events.
- Added debug visualization for collision boundaries to speed up iteration and testing.

Game-Side Implementation:

- Configured consistent `CollisionComponent` setups for player and enemy entities to match their visual hitboxes.
 - Implemented specialized collision responses for gameplay (e.g., player attacks, enemy hit reactions).
 - Applied collision layers for nuanced interactions (e.g., player projectiles ignore certain obstacles but hit enemies).
 - Managed invulnerability frames post-collision to prevent rapid successive damage.
 - Used variable-sized hitboxes for different attack animations to precisely match visuals.
-

2. Animation

I deserve a mark of **93%** for animation.

I demonstrate competency in this area in the game I made by:

Engine-Side Implementation:

- Developed a memory-efficient animation pooling system reusing **AnimationClipConfig** objects across entities.
- Built a frame-based animation system with configurable playback rates and anchor points.

- Implemented animation callbacks (**onFinished**, **onFrame**) to trigger gameplay logic at precise frames.
- Added directional flipping support without duplicating assets.
- Integrated animations with ECS via **AnimationComponent** and **AnimationSystem**.

Game-Side Implementation:

- Player transitions smoothly between idle, run, jump, attack, and hit animations via state machine triggers.
- Enemy animations are synchronized with AI behaviors (patrol, chase, attack).
- Attack animations time hit detection windows to specific frames for accurate gameplay feel.
- State transitions use animation callbacks to align logic execution with visuals.
- Applied consistent anchor modes to maintain visual alignment during rapid state changes.

3. Physics or Networking

I choose to be evaluated on: ☒ **Physics** ☐ **Networking**

I deserve a mark of 90% for this area.

I demonstrate competency by:

Engine-Side Implementation:

- Created a component-based physics system separating horizontal (**MoveComponent**) and vertical (**JumpComponent**) motion.
- Implemented gravity, friction, and velocity updates scaled to frame time (Δt).
- Supported impulse-based movement for dashes, jumps, and knockbacks.
- Structured the physics update pipeline to process input, calculate movement, resolve collisions, and finalize positions in sequence.
- Built a ground detection system to accurately determine when entities are supported by surfaces.

Game-Side Implementation:

- Player movement features responsive controls with configurable acceleration, deceleration, and max speed.
- Environmental interactions (platforms, obstacles) correctly influence moving entities.

Reviewer Acknowledgment

I had this self-evaluation reviewed by: _____ **Meng Teng** _____

Game Engine Course Final Project – Self-Evaluation

Student Name: Haolun Yuan

Project Title: FinalProject

Date: 2025/08/13

I, Haolun Yuan, deserve a mark of 95% for final project. I demonstrate competency in this area in the game engine I made by building a game engine with the following features from scratch:

Engine Overview

- Modern 2D game engine built on Entity-Component-System (ECS) architecture.
- Modular design with clean separation of concerns.
- Efficient memory management and comprehensive tooling.

Core Architecture

- Entity-Component-System (ECS):
 - Uses entt for high-performance, cache-friendly data management.
 - Scene Management:
 - Centralized ownership model.
 - Safe entity creation and component access.
 - System Pipeline:
 - Deterministic update order.
 - Lifecycle phases: init → update → shutdown.
 - Memory Model:
 - Explicit ownership via Scope/Ref patterns.
 - Current: std::smart_ptr.
 - Planned: intrusive counting reference system.
-

State Machine System

- Component-Based FSM for flexible entity behavior.
 - State Transitions:
 - Event-driven.
 - Configurable conditions and triggers.
 - Entry/Exit Actions:
 - Hooks for logic execution during state changes.
 - Hierarchical States:
 - Parent-child state relationships.
 - Inherited behavior.
-

Renderer2D System

- Batch Rendering:
 - Optimized sprite batching.
 - Fewer draw calls for better performance.
- Camera System:
 - Orthographic and perspective support.

- Viewport management.
 - Texture Management:
 - Efficient loading, caching, and atlas support.
 - Primitive Drawing:
 - Debug tools for lines, rectangles, circles.
 - Shader System:
 - Customizable shader pipeline.
 - Uniform management.
 - Layer System:
 - Z-ordering and visibility control.
-

Resource Management

- Asset Pipeline:
 - Streamlined import & management of textures, audio, etc.
 - Resource Caching:
 - Prevents redundant asset loads.
 - Hot Reloading:
 - Runtime asset updates for rapid iteration.
 - Type-Safe Handles:
 - Strongly typed resource references.
-

Input System

- Device Abstraction:
 - Unified keyboard, mouse, and controller input.
 - Input Mapping:
 - Configurable action-to-input binding.
 - Context-Sensitive Controls:
 - Different schemes based on game state.
 - Input Buffering:
 - Optional command queuing for responsiveness.
-

Event System

- Event Bus:
 - Decoupled publish-subscribe communication.
 - Typed Events:
 - Compile-time type safety.
 - Deferred Events:
 - Trigger in future frames.
-

Scene Management

- Scene Hierarchy:
 - Parent-child entity relationships.
 - Scene Transitions:
 - Load, unload, transition between scenes.
 - Serialization:
 - Save/load with JSON or binary (WIP).
-

Editor Tools (WIP)

- Scene Hierarchy Panel:
 - Visual organization and selection.
- Component Inspector:
 - Real-time property modification.
- State Machine Editor:
 - Visual graph-based creation/editing.
- Debug Visualization:
 - System states, performance metrics.

Utility Systems

- Logger:
 - Multiple severity levels.
 - Channel filtering.
- Profiler:
 - Visual performance monitoring.
 - Bottleneck detection.

Reviewer Acknowledgment

I had this self-evaluation reviewed by: Meng Teng